DATABASE DESIGN DOCUMENT

Database Purpose

The TrackWIL database stores everything the application needs to track Work-Integrated Learning logbooks: the accounts of students, coordinators and lecturers; each student's WIL placements with their required hours; the daily logbook entries of each placement with their hours, evidence file name and verification status; the details of each uploaded WIL document; and the one-time verification links that workplace mentors use to approve or reject entries.

The database must support these queries efficiently and reliably:

- Sign-in: find a user by email address (case-insensitive).
- Student logbook: list a placement's entries, newest first, total its verified hours and work out whether the placement is completed.
- Documents: list a placement's documents and find the document already in a slot (a type, or a type and semester) so that it can be replaced.
- Verification: find a link by its token, check that it is unused and unexpired, and record the mentor's decision on the link and the entry together.
- Monitoring dashboard: for every student, find the current placement, count entries, pending verifications and missing documents, total verified hours and find the latest verification date.
- Evidence download: find the entry that owns an evidence file to check who may open it.

SQLite was chosen because it is a full relational database (keys, constraints and transactions) that runs inside the application through Node.js's built-in node:sqlite module, so no database server has to be installed. The database is a single file, data/trackwil.db, which is created and seeded automatically when the application first starts.

Entity-Relationship Diagram

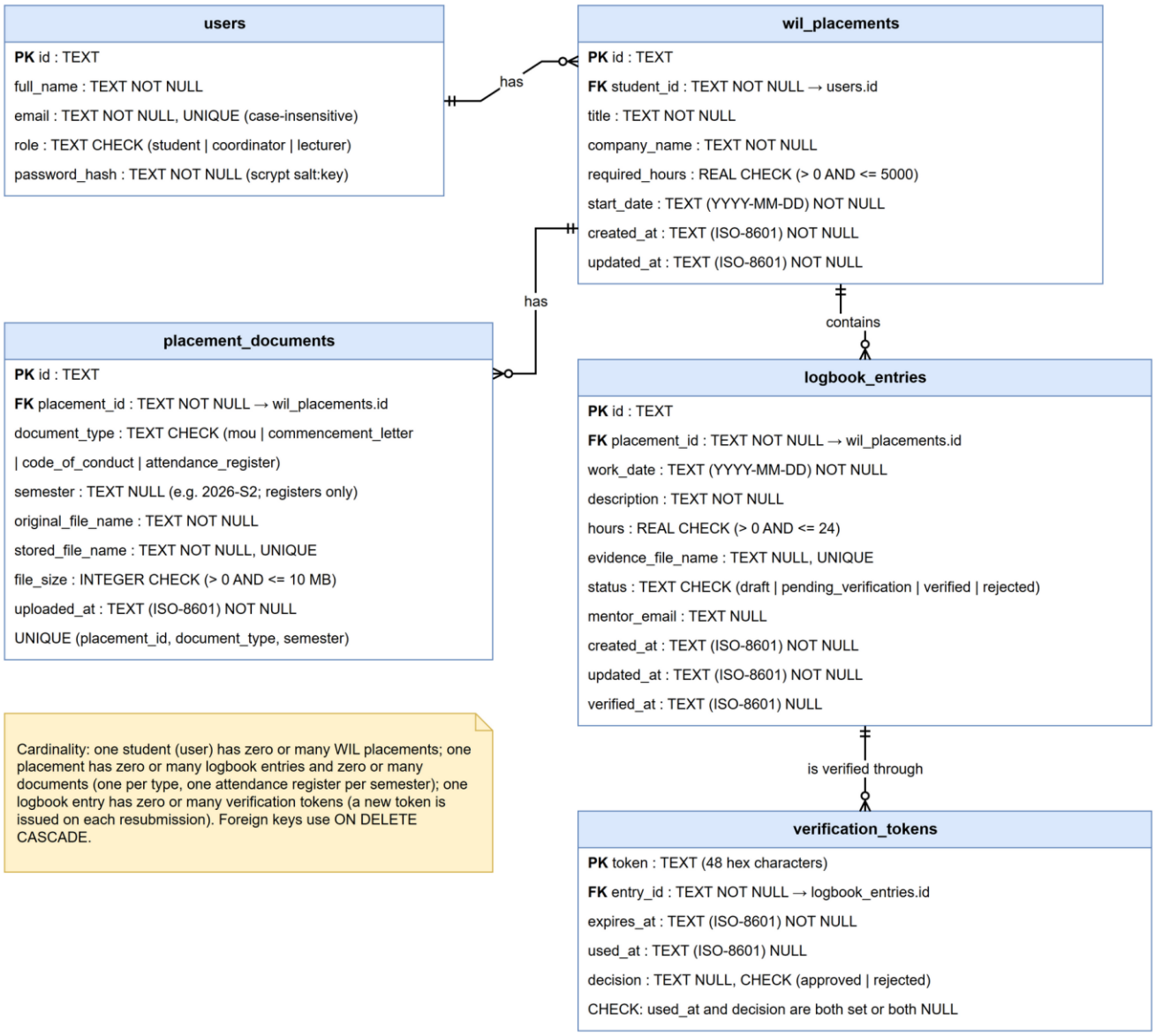


Figure 1: Entity-relationship diagram

Table 1: Relationships and cardinalities

Relationship	Cardinality	Implemented by	Meaning
users to wil_placements	One to zero-or-many	wil_placements.student_id references users.id	A student can have any number of WIL placements; each placement belongs to exactly one student
wil_placements to logbook_entries	One to zero-or-many	logbook_entries.placement_id references wil_placements.id	A placement can have any number of entries; each entry belongs to exactly one placement (and so to one student)
logbook_entries to verification_tokens	One to zero-or-many	verification_tokens.entry_id references logbook_entries.id	An entry gets a new link each time it is submitted (including resubmission after rejection); each link belongs to exactly one entry
wil_placements to placement_documents	One to zero-or-many	placement_documents.placement_id references wil_placements.id	A placement has at most one document of each type and one attendance register per semester; each document belongs to exactly one placement

Mentors are not stored as a separate entity because they have no account in version 1.0. The mentor is identified by the email address stored on each entry, and each verification is recorded on the token that the mentor used.

Data Dictionary

All text values are stored as TEXT and numbers as REAL (file sizes as INTEGER), which are SQLite's storage classes. Dates and times are stored as ISO-8601 text in UTC (for example 2026-10-22T08:00:00.000Z), which sorts correctly as text.

Table 2: users

Column	Type	Size / format	Null	Key	Constraint and description
id	TEXT	Up to 44 characters (e.g. student-<UUID>)	No	PK	Unique account identifier
full_name	TEXT	Up to about 120 characters	No		The user's full name, shown in headers and the dashboard
email	TEXT	Up to about 150 characters	No	Unique	Sign-in name; UNIQUE with COLLATE NOCASE so letter case does not matter
role	TEXT	student, coordinator or lecturer	No		CHECK limits the value to the three roles
password_hash	TEXT	32-character salt, a colon and a 128-character key	No		scrypt hash of the password; the password itself is never stored

Table 3: wil_placements

Column	Type	Size / format	Null	Key	Constraint and description
id	TEXT	UUID (36 characters)	No	PK	Unique placement identifier
student_id	TEXT	As users.id	No	FK	References users(id) ON DELETE CASCADE; indexed with start_date
title	TEXT	Up to about 60 characters	No		The placement's name, for example WIL 1
company_name	TEXT	Up to about 120 characters	No		The host company
required_hours	REAL	More than 0, at most 5 000	No		Hours this placement requires; CHECK (required_hours > 0 AND required_hours <= 5000)
start_date	TEXT	YYYY-MM-DD	No		The day the placement started; also decides which semesters need an attendance register
created_at	TEXT	ISO-8601 UTC	No		When the placement was added
updated_at	TEXT	ISO-8601 UTC	No		When the placement last changed

Table 4: logbook_entries

Column	Type	Size / format	Null	Key	Constraint and description
id	TEXT	UUID (36 characters)	No	PK	Unique entry identifier

Column	Type	Size / format	Null	Key	Constraint and description
placement_id	TEXT	As wil_placements.id	No	FK	References wil_placements(id) ON DELETE CASCADE; indexed with work_date
work_date	TEXT	YYYY-MM-DD	No		The day the work was done
description	TEXT	Free text	No		What the student did
hours	REAL	More than 0, at most 24	No		CHECK (hours > 0 AND hours <= 24)
evidence_file_name	TEXT	UUID plus extension	Yes	Unique	Name of the stored evidence file, or NULL if none
status	TEXT	draft, pending_verification, verified, rejected	No		CHECK limits the value; default draft
mentor_email	TEXT	Email address	Yes		The workplace mentor for this entry; required before submission
created_at	TEXT	ISO-8601 UTC	No		When the entry was created
updated_at	TEXT	ISO-8601 UTC	No		When the entry last changed
verified_at	TEXT	ISO-8601 UTC	Yes		When the mentor approved the entry

Table 5: verification_tokens

Column	Type	Size / format	Null	Key	Constraint and description
token	TEXT	48 hexadecimal characters	No	PK	Random value from 24 bytes (192 bits); appears in the link /verify/<token>
entry_id	TEXT	As logbook_entries.id	No	FK	References logbook_entries(id) ON DELETE CASCADE; indexed
expires_at	TEXT	ISO-8601 UTC	No		7 days after the link was created
used_at	TEXT	ISO-8601 UTC	Yes		When the mentor used the link; NULL while unused
decision	TEXT	approved or rejected	Yes		The mentor's decision; CHECK requires used_at and decision to be both set or both NULL

Table 6: placement_documents

Column	Type	Size / format	Null	Key	Constraint and description
id	TEXT	UUID (36 characters)	No	PK	Unique document identifier
placement_id	TEXT	As wil_placements.id	No	FK	References wil_placements(id) ON DELETE CASCADE
document_type	TEXT	mou, commencement_letter, code_of_conduct or attendance_register	No		CHECK limits the value to the four WIL documents
semester	TEXT	YYYY-S1 or YYYY-S2	Yes		Only for attendance registers; CHECK requires a semester for a register and none for the other documents
original_file_name	TEXT	Up to 200 characters	No		The name of the uploaded file, shown to users
stored_file_name	TEXT	UUID plus extension	No	Unique	Name of the file in data/uploads/documents
file_size	INTEGER	Bytes	No		CHECK (file_size > 0 AND file_size <= 10485760), at most 10 MB
uploaded_at	TEXT	ISO-8601 UTC	No		When the file was uploaded or last replaced

Normalisation

First normal form (1NF). Every column holds one value and every table has a primary key. For example, each day of work is its own row in logbook_entries rather than a list of days stored in one column of users.

Second normal form (2NF). Every table has a single-column primary key, so no column can depend on only part of a key.

Third normal form (3NF). No non-key column depends on another non-key column. A student's name is stored once in users and a placement's required hours once in wil_placements, not repeated in every entry, so changing them updates every screen at once (no update anomaly). An entry stores only its placement_id and not the student, because the student is found through the placement; storing both could let them disagree. Each token row holds only facts about that link.

Calculated values are not stored. Verified hours, whether a placement is WIL completed and each student's status are calculated from the entries every time a page is shown. Storing them would create duplicate data that could become wrong when an entry changes.

Change from the prototype. The first prototype stored each collection as a JSON file and rewrote the whole file on every change, with no keys or constraints. Moving to SQLite added enforced primary and foreign keys, CHECK constraints and transactions without changing the shape of the data.

Security and Integrity

- **Constraints: primary keys, UNIQUE (email, evidence file name, stored document name), a unique index that allows one document of each type per placement and one attendance register per semester, foreign keys with PRAGMA foreign_keys = ON, and CHECK constraints for roles, statuses, hours, required hours, document types, semesters, file sizes and the token decision.**
- **Transactions: submitting an entry (insert token and set Pending), recording a mentor decision (mark token used and update the entry) and saving a document (find its slot, then insert or replace) each run in a single transaction,** so a failure cannot leave a half-finished change. The token is marked used only WHERE used_at IS NULL, so a link cannot be used twice.
- **Validation:** every value is validated in the Server Action before it reaches the database; the constraints are a second line of defence.
- **SQL injection:** every query uses prepared statements with ? placeholders; user input is never joined into SQL text.
- **Access control:** only the repository module runs SQL, and it is called only from server code after the user's role and ownership have been checked.
- **Sensitive data:** passwords are stored only as salted scrypt hashes. Evidence files and WIL documents are stored outside the public web folder under random names and served only to their owner and to staff; a document is accepted only if its content matches its file type.
- **Backups and recovery:** the database is a single file that can be copied while the application is stopped. For the demonstration, npm run db:reset rebuilds it from the scripts.
- **Privacy:** only synthetic data is stored until written campus permission is granted (POPIA).

SQL Scripts

Table 7: Database scripts in the repository

Script	Location	Purpose
schema.sql	05_Source_Code/trackwil/database/schema.sql	Creates the five tables, keys, constraints and indexes
seed.sql	05_Source_Code/trackwil/database/seed.sql	Inserts synthetic demonstration users, placements, entries, links and document records, with dates relative to the current date
demo-documents/	05_Source_Code/trackwil/database/demo-documents	Synthetic PDFs that the seed data refers to
reset-db.mjs	05_Source_Code/trackwil/scripts/reset-db.mjs	Deletes and rebuilds the database from the two scripts and restores the demo documents (npm run db:reset)
db.ts	05_Source_Code/trackwil/src/lib/db.ts	Applies the schema on start-up and seeds an empty database
repository.ts	05_Source_Code/trackwil/src/lib/repository.ts	All queries and transactions used by the application